

12.高级数据结构

1.

已知广义表数据结构如下：

```
typedef enum {ATOM, LIST} ElemTag;
typedef struct GLNode {
    ElemTag tag;
    union {
        char atom; //原子
        struct {
            struct GLNode *hp, *tp;
        } ptr; //表头，表尾
    } un;
};
```

请你给出伪代码，检测一个表是否为纯表

算法思想：检查是否有重复出现的原子存在

```

// 全局/局部集合，用于检测重复访问
visitedList = empty set of GLNode*
visitedAtom = empty set of char

function isPureRecursive(L):
    // 递归检测：广义表是否有环且原子不重复

    if L == NULL:
        return true // 空节点是有效的

    if L->tag == ATOM:
        // 原子节点：检查是否重复出现
        atom = L->un.atom
        if atom in visitedAtom:
            return false // 原子重复，无效
        else:
            add atom to visitedAtom
            return true // 原子首次出现，有效
    else:
        // 表节点：检查是否形成环
        if L in visitedList:
            return false // 表节点重复访问，有环
        else:
            add L to visitedList // 标记当前表节点已访问

            // 递归检查表头和表尾
            head = L->un.ptr.hp
            tail = L->un.ptr.tp

            if not isPureRecursive(head) or not isPureRecursive(tail):
                return false
            else:
                return true

function isPure(L):
    // 主函数：检测广义表是否满足条件
    // 条件1：原子不重复
    // 条件2：无环（表节点不重复访问）

    clear visitedList
    clear visitedAtom

```

```
return isPureRecursive(L)
```

2.

CSR

格式存储的稀疏矩阵如下：

```
class CSR
{
    int row, col, nnz;    // #行数、列数、非零数个数
    double value[nnz];    // 非零元素数组
    int row_ptrs[row+1]; // 每行非零元素在value[]中的起始索引
    int col_index[nnz];   // 非零元素所在列索引
};
```

计算 $y=Ax$ 的伪代码

```
# 计算  $y = Ax$ ，其中  $A$  是 CSR 格式的稀疏矩阵， $x$  是向量， $y$  是结果向量
function multiplyCSR(A, x):
     $y = \text{new array of size } A.\text{row}, \text{ initialized to } 0$ 
    for  $i$  in  $0$  to  $A.\text{row} - 1$ :
        for  $j$  in  $A.\text{row\_ptrs}[i]$  to  $A.\text{row\_ptrs}[i+1] - 1$ :
             $y[i] += A.\text{value}[j] * x[A.\text{col\_index}[j]]$ 
    return  $y$ 
```

3.

```
      38
     /  \
    20    60
   /  \  /  \
  10 25 50 70
   \  \  \
  15  30 55
```

中序遍历：10, 15, 20, 25, 30, 38, 50, 55, 60, 70 (

4.

Trie 树的空间开销

- 节点结构：每个节点需要存储一个大小为字符集 Σ 的指针数组（或类似索引结构），用于指向可能的子节点。
- 问题：当字符集较大时（如 Unicode），每个节点的指针数组会占用大量空间，且多数指针为空，导致空间利用率低。
- 示例：若字符集为 ASCII（256 个字符），每个节点就有 256 个指针，即使实际存在的子节点很少。

Merkle Patricia Tree (MPT) 的空间优化

- MPT 结合了 Patricia Trie 和 Merkle 树 的特点：
- 路径压缩：将只有一个子节点的节点与其父节点合并，存储为一个节点（包含多个字符的前缀），显著减少节点数量。
- 节点类型多样化：扩展节点存储共享前缀和下一个节点的哈希；分支节点最多 16 个子节点，仅存储实际存在的子节点指针；叶子节点存储键值对。
- 哈希链接：每个节点通过哈希值引用，便于验证但引入额外存储

综上，MPT 通过路径压缩和多样化节点结构，有效减少了 Trie 树在存储大量键值对时的空间开销，其哈希适用于稀疏数据的情况

其他优化

- 自适应节点结构：根据子节点数量动态选择存储方式，节点少用哈希，多了就用数组
- 使用外部存储或压缩技术，将不常用部分存储在磁盘，或对节点进行压缩